

Prognostisering av Bergen Bysykkel

Rapporten dokumenterer arbeidet med å predikere antall ledige sykler på en stasjon én time fram i tid for ni utvalgte stasjoner. Datasettet kombinerer tre kilder: `stations.csv`, `trips.csv` og `weather.csv`. Alle tidsstempler behandles i UTC og aggregeres til hele timer. Vi lager et timesrutenett per stasjon og bruker LOCF for å fylle `free_bikes` på hver hele time. I tillegg lager vi lagg og glidende gjennomsnitt av `free_bikes`, `trips`-signaler og tidsvariabler. Siden perioden april til august 2024 har svært svak dekning for `free_bikes` ekskluderer vi perioden før modellering slik at hver rad fortsatt representerer en gyldig prediksjon for neste time.

I Del 1 bruker vi treningsdata til å beskrive datastruktur, mangler og fordelingsegenskaper, og vi undersøker mønstre. Her finner vi bl.a. mønster for rushtider og sammenheng mellom `trips` og endring i tilgjengelige sykler. I Del 2 deler vi data kronologisk, imputerer manglende verdier med medianer fra treningssettet, og sammenligner flere regresjonsmodeller mot en enkel LOCF-baseline. Modellvalg gjøres på valideringssettet, før beste modell trenes på train og val og evalueres én gang på test for å måle generalisering.

Til slutt i Del 3 gjør vi løsningen kjørbart for en bruker gjennom skriptene `train.py` og `predict.py`. `train.py` trener, velger og lagrer modellen, mens `predict.py` henter siste tilgjengelige tidsstempel fra rådata (`stations.csv`), bygger features ved neste hele time og produserer heltallsprediksjoner for én time fram i tid i lokal Bergen-tid (Europe/Oslo). Sammenstillingen sikrer at resultatene kan reproduseres når nye rader legges til i rådata, og at prediksjonene som skrives ut er konsistente med valg og antagelser fra EDA og modelleringen.

- Fremgangsmåte og begrunnelser

Vi starter med å importere relevante filer og pakker som skal brukes for oppgaven. Jeg har også inkludert en kort (generert) kode som skal sørge for at koden kan kjøres av andre uten problemer.

Etter å ha lest dataen kunne vi se at det i perioden april til august 2024 er ekstremt svak dekning av data for `free_bikes`, og ved å inkludere dette intervallet med i sluttregningen mottar vi feilaktig relasjon og konklusjon. På grunn av dette har jeg valgt å utelukke denne delen av datasettene totalt, og heller jobbe med alt etter den perioden for å få mest mulig realistisk prediksjon. Vi ekskluderte intervallet før

modellering, og krevde at label `y_t_plus_1h` kun beholdes når neste time faktisk finnes i datasettet. Dette sikrer at én rad fortsatt tilsvarer én gyldig prediksjon.

Vi predikerer antall ledige sykler `y_t_plus_1` for 9 utvalgte stasjoner i Bergen. Tidsoppløsninger er gjort timevis i UTC. Evalueringsoppsettet er gjort kronologisk 70/15/15 (train/val/test) for å unngå informasjonslekkasje.

Tidsperioden er dekket fra 2024-04-10 23:00:00+00:00 til 2025-05-02 15:00:00+00:00 (UTC) med 5847 unike timetrinn og totalt 52622 antall rader fordelt per split (36834 - train/7894 - val/7894- test). Hver rad er et tidspunkt for én stasjon, og tilsvarer én prediksjon av `y_t_plus_1h`.

Vi kobler sammen data fra stations-, trips- og weather-filene på timesnivå og bygger et felles rutenett per stasjon. All tid konverteres til UTC for å sikre sammenlignbare tidssteg.

Labelen er satt til `y_t_plus_1h`. Som baseline bruker vi LOCF som fungerer greit utenom rushtider, og gjør at tidsfeatures og trafikkfeatures blir viktige.

Nedbør aggregeres som sum/time, temperatur og vind som middel/time. Vi konstruerer departures og arrivals i tillegg til å lage rolling windows på 3 timer, og legger til `hour_of_day`, `day_of_week`, `is_weekend` samt glatting av `free_bikes` (lag1, ma3).

Etter dataen er renset bruker vi en pipeline for å samle dataen i `model_ready.csv`, som vi kan bruke videre i `train.py` for å lage, finne og lagre beste modell ved å teste dataen på ulike modeller. Den beste modellen er den med lavest RMSE på valideringsdata. Etter vi har funnet den kan vi lagre den med `pickle`.

I `predict.py` leser vi alltid siste rådata, finner siste tidsstempel (i `stations.csv`), runder opp til neste hele time og lager features ved `t` på samme måte som i trening. Vi fyller eventuelle mangler med treningsmedianer og lar den lagrede modellen predikere neste time. Prediksjonene blir presentert som avrundet heltall, klippet til å være minst 0. Til slutt skriver vi ut siste tidsstempel fra rådata, både direkte og avrundet, og prediksjonstidspunktet (begge i lokal Bergen-tid). Vi skriver ut en tabell som viser nåværende og predikerte antall sykler for de ni stasjonene vi er interesserte i.

- Dataegenskaper (struktur, feil, mangler, innsikter) - Relevante visualiseringer

Kilder

Vi bruker tre kilder: stations.csv (tilgjengelighet), trips.csv (avgang/ankomst) og weather.csv (temperatur, vind, nedbør). Vi fokuserer på de 9 stasjonene som er listet i oppgaveteksten. Én rad = [station, time] med mål `y_t_plus_1h`, features.

Struktur

Tid er håndtert i UTC, manglende tidsstempler er adressert, og det er ingen duplikater i (station, time)

Beskrivende statistikk av treningsdata:

	count	mean	std	min	25%	50%	75%	max
free_bikes	36835.0	5.490	6.914	0.0	0.000	2.000	9.000	34.0
temperature	36421.0	5.353	5.977	-12.6	1.700	5.400	9.400	27.9
precipitation	36835.0	0.334	0.743	0.0	0.000	0.000	0.300	6.8
wind_speed	36466.0	11.267	7.938	0.0	5.000	9.200	15.900	40.6
hour_of_day	36835.0	11.483	6.922	0.0	5.000	11.000	17.000	23.0
day_of_week	36835.0	2.994	2.015	0.0	1.000	3.000	5.000	6.0
departures	36835.0	0.636	1.337	0.0	0.000	0.000	1.000	17.0
arrivals	36835.0	0.666	1.407	0.0	0.000	0.000	1.000	18.0
departures_3h	36835.0	1.908	3.182	0.0	0.000	1.000	3.000	30.0
arrivals_3h	36835.0	1.997	3.398	0.0	0.000	1.000	3.000	33.0
net_flow	36835.0	0.030	1.380	-13.0	0.000	0.000	0.000	14.0
net_flow_3h	36835.0	0.089	2.775	-22.0	0.000	0.000	1.000	26.0
free_bikes_lag1	36835.0	5.491	6.915	0.0	0.000	2.000	9.000	34.0
free_bikes_lag2	36835.0	5.493	6.916	0.0	0.000	2.000	9.000	34.0
free_bikes_lag3	36835.0	5.495	6.916	0.0	0.000	2.000	9.000	34.0
free_bikes_ma3	36835.0	5.492	6.833	0.0	0.333	2.333	8.667	34.0
y_t_plus_1h	36835.0	5.488	6.913	0.0	0.000	2.000	9.000	34.0

Statistikken beskriver at `y_t_plus_1h` har ca samme fordeling som `free_bikes`. Trips variablene, `departures` og `arrivals`, er høyreskjeve. Netto flyt er sentrert rundt 0. Værvariablene er variert, mens nedbør er høyreskjev. Laggvariabler følger nivået i `free_bikes` og finner trender.

Manglende data (andel NAN) per kolonne i treningsdata:

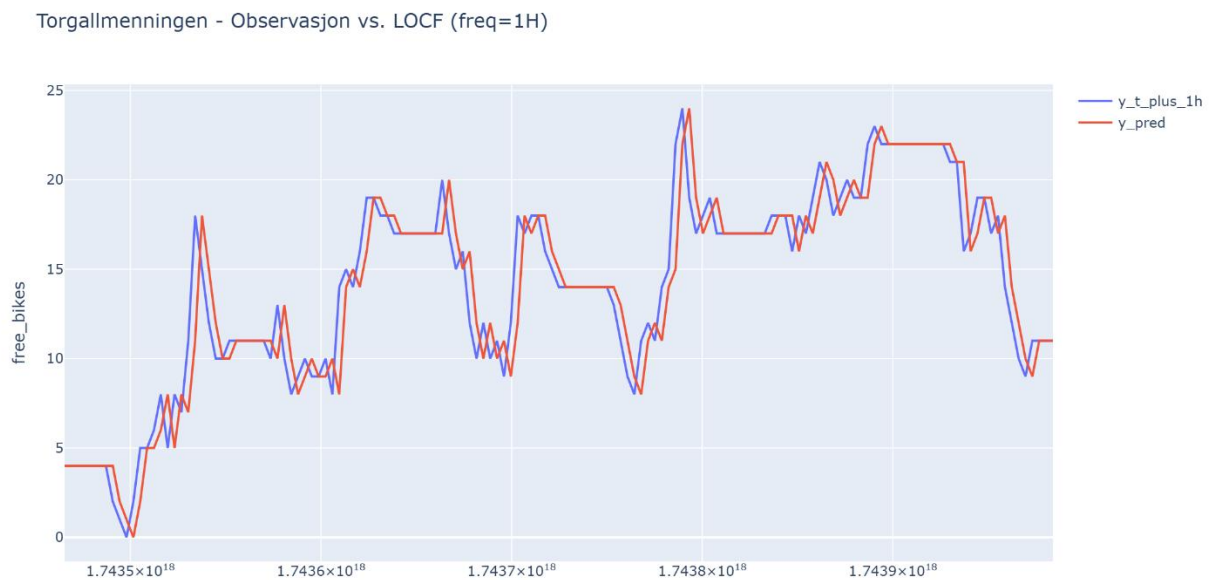
	missing_rate
temperature	0.0112
wind_speed	0.0100

Andel manglende data på treningsdata er lav. Det er noe manglende data for temperatur og vind, og i modelleringen imputerer vi dette med train-median.

Relevante visualiseringer

All datautforskning (utenom observasjon vs. LOCF) er gjort på treningsdata etter sklearn sin `train_test_split` fordelt 70/15/15 med random seed 42. Val/test beholdes for modellvalg og endelig evaluering senere.

Observasjon vs. LOCF:

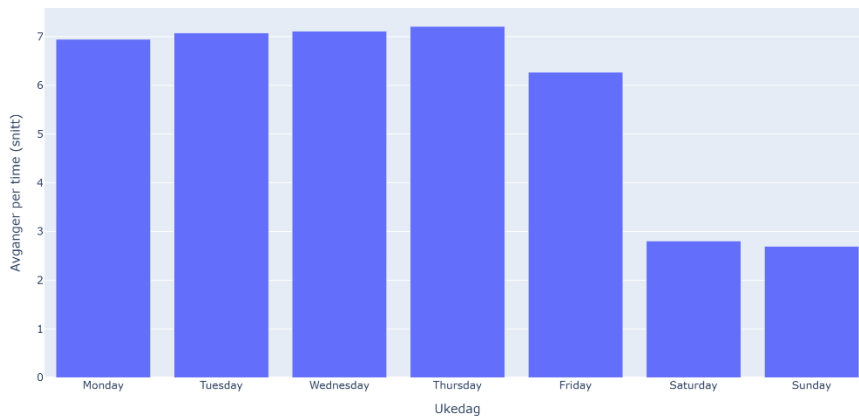


LOCF ($y(t)$) følger fasiter ($y(t+1)$) som den skal hele veien, men når observasjonsnivået stiger underpredikerer LOCF (med ett steg). På samme måte overpredikerer LOCF når observasjonsnivået faller. Når kurven er flat ligger LOCF rett på.

Konklusjon: Vi bør ha tidsfeatures og trafikkfeatures for å få en nøyaktig prediksjon.

Gjennomsnittlige totale avganger per ukedag:

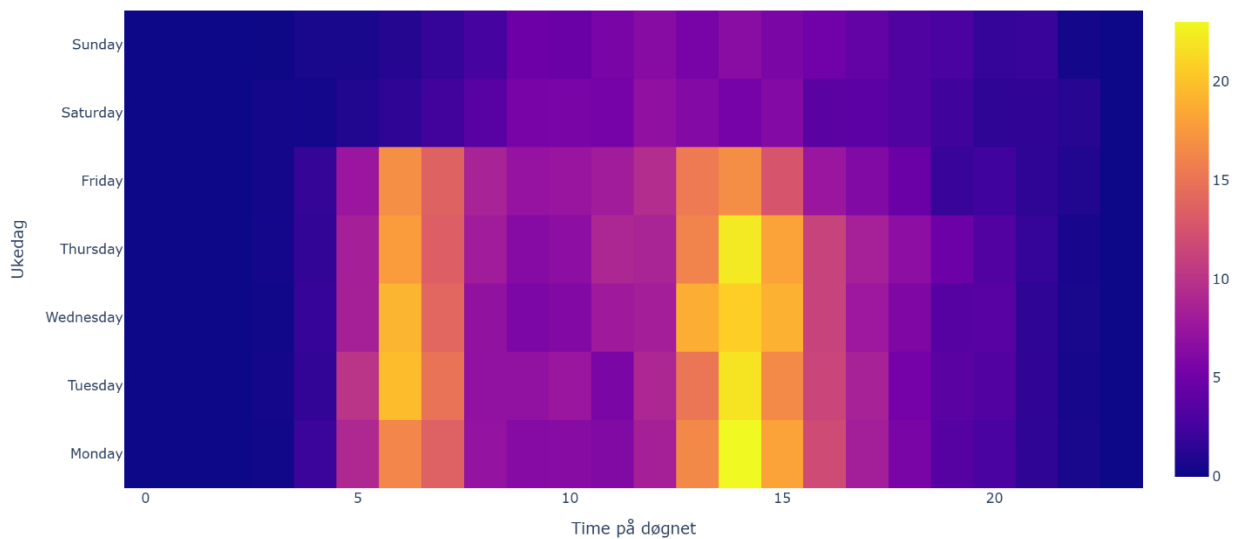
Gjennomsnittlige totale avganger per ukedag



Avganger per time i gjennomsnitt er høyere i ukedager enn i helger. Dette betyr at ukedager og helger som egen feature vil gi en bedre prediksjon.

(Heatmap) Totale avganger: ukedag og time:

Totale avganger: ukedag og time (gjennomsnitt)



Det er tydelige topper i avganger rundt kl. 06 og 14 på hverdager. Dette er da enda et tegn på at time på døgnet og ukedager er viktige for prediksjonen.

Gjennomsnittlig antall ledige sykler per time, per stasjon:

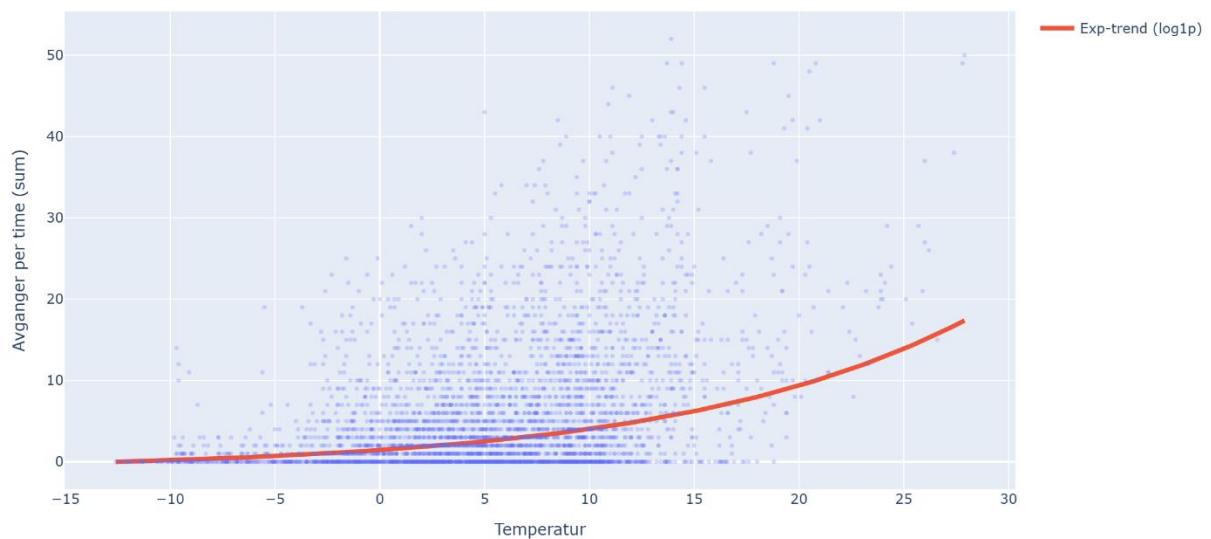
Gjennomsnittlig ledige sykler per time, per stasjon



Stasjonene vi er interesserte i har ulik tilgjengelighet. På grunn av dette kan det være greit å bruke stasjons-IDer og muligens kapasiteten når vi skal predikere.

Totale avganger vs. temperatur (med eksponentiell trend):

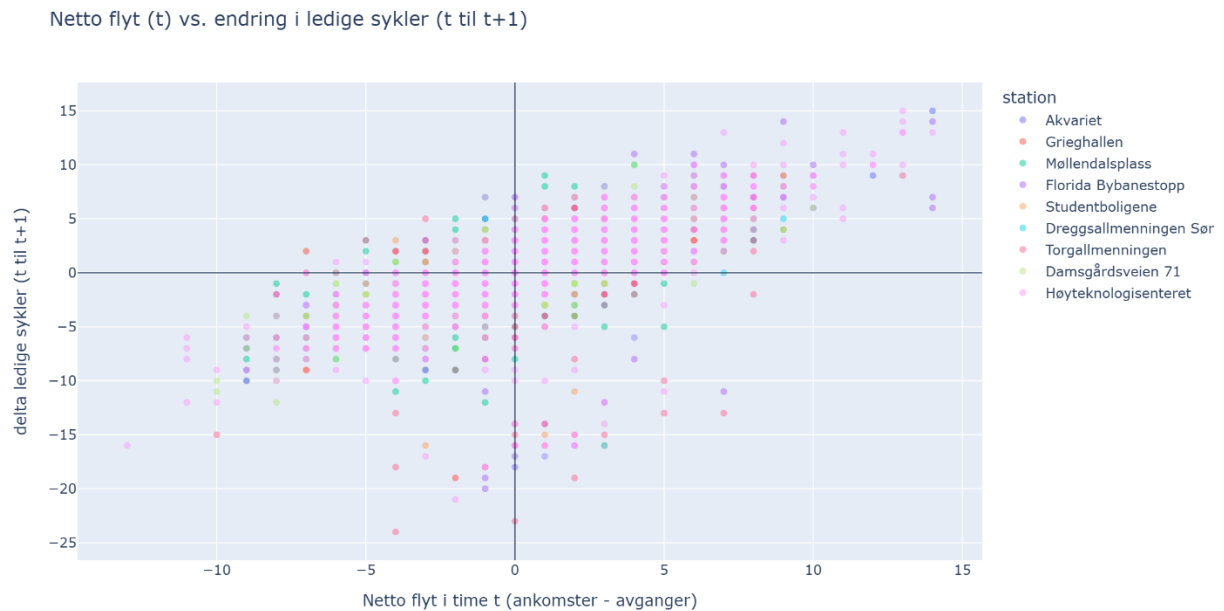
Totale avganger vs. temperatur - eksponentiell trend



Det er flere turer ved høyere temperatur. Den eksponentielle trenden er svak, men viser fortsatt en

stabil økning. Været kan bidra til flere turer, men har ikke stor påvirkning. Vi beholder vær når vi predikerer, men effekten er liten.

Netto flyt i time t vs. endring i ledige sykler:



Positive netto-flyt signaler har en sammenheng med hvor mange sykler som er ledige neste time. Dette gjør at vi kan si at trips-signalene har betydning for målet.

Median endring i ledige sykler per nettoflyt:

Høyere nettoflyt gir systematisk større endring i neste time, og vice versa. Denne effekten er sterk selv etter aggregering og vi bør bruke den i modellen.

Visualiseringer som ikke er inkludert

Jeg har valgt å la være å bruke flere av visualiseringene da ikke alle er like meningsfulle. Resten av visualiseringene ligger under reports/figures, eller ved å kjøre EDA.ipynb.

- Preprosesseringstiltak og valg

Time-grid og LOCF

Per stasjon tar vi med siste observasjon frem til heltimespunkter

Label

$y_{t_plus_1h} = \text{free_bikes}(t+1h)$

Trips features

departures, arrivals, rolling 3t (unngår lekkasje)

Vær

Timeresample – mean for temperatur og vind, sum for nedbør

Tids features

hour_of_day, day_of_week, is_weekend

Lagg

free_bikes_lag1, glidende snitt ma3

Sikring

Har laget assertions for unikhet, labeljustering og verdiområder.

Begrunnelse

- LOCF som enkel baseline
- Rolling windows og laggs fanger treghet og trend
- Ukedag/time of day og trips gir rushtidssignal

- Modellutvalg og evaluering

Vi delte data kronologisk, først 15% til test, så splittet vi resten til 70% train og 15% val. Baseline LOCF ga $RMSE_{test} = 1.505$. Vi sammenlignet med ElasticNet, RandomForest, HistGradientBoosting og SVR. Vi valgte modellen med lavest val-RMSE. Den beste modellen av de vi trente var SVR med val-RMSE på 1.060673. Etter «retrening» på train og val fikk vi test-RMSE=1.461, mot baseline = 1.505. Dette er en forbedring på ca. 3%.

```
Baseline RMSE (LOCF) på hele settet: 1.505
Størrelser: 36829 7892 7892
      rmse_train  rmse_val
SVR      1.076404  1.060673
ElasticNet 1.162704  1.062978
HistGBR    0.976745  1.098135
RandomForest 0.416157  1.120622
Beste på val: SVR
Baseline RMSE (LOCF): 1.505
SVR test-RMSE: 1.461
```

- Variabelutvinning og manglende data

Features vi har med inkluderer tidsvariabler (hour_of_day, day_of_week, is_weekend), vær (temperature, wind_speed, precipitation), trips-signaler (departures, arrivals, rolling windows og net_flow), lag (free_bikes_lag1-3/ma3) og en dummy variabel for station.

Manglende verdier fylles inn med median fra treningssettet og samme verdier brukes på val/test.

- Modellering

Vi modellerer et regresjonsproblem der hver modell ble trent på train, sammenlignet på val (RMSE), og kun den beste modellen ble trent på val og train og evaluert én gang på test. Baseline (LOCF) er inkludert for referanse. Resultatene viser at SVR generaliserer best av de modellene vi trente.

- Sammenstilling og deployment

Målet for denne delen var å gjøre løsningen robust og kjørbart for en bruker gjennom to skript hvor ett trener og lagrer modellen og hvor den andre gjør prediksjon på rådata. I `train.py` leser vi inn råkildene (`stations.csv`, `trips.csv` og `weather.csv`), lager features på time-nivå for hver målstasjon, og definerer målet som antall ledige sykler én time frem i tid (`y_t_plus_1h`). Vi bruker LOCF for å predikere `free_bikes` på hver heltime tidsstempel, laggt- og glidende gjennomsnitt for `free_bikes`, `arrivals` og `departures` timesoppsummert med 3-timers vindu for å dekke lekkasje, vær på timesnivå, og enkle tidsvariabler som time på døgnet, ukedag og helgeindikator. Stasjonsnavn kodes som dummykolonner.

I `predict.py` gjenskaper vi nøyaktig samme oppbyggingen av features, men nå ved siste tilgjengelige tidspunkt i rådata. Vi finner først siste tidsstempel på tvers av alle kildene, runder opp til neste hele time, og lager features for dette tidspunktet (`t`) for hver målstasjon. Deretter aligner vi dummykolonner og kolonnerekkefølge mot det som ble brukt i trening, fyller manglende data med de lagrede treningsmedianene, og lar den lagrede modellen predikere $y(t+1)$. Til slutt runder vi prediksjonene til heltall og klipper dem til å være minst null. Alle tider presenteres i lokal Bergen-tid, som er samme tidssone som Europe/Oslo. Utskriften viser siste tidsstempel i data, neste hele klokke-time, tidspunktet prediksjonen gjelder for, samt en tabell med nåværende og predikerte antall sykler for hver målstasjon.

- Resultater

Ved kjøring av `predict.py` produserer systemet prediksjoner for den neste hele timen basert på det siste tilgjengelige tidsstempelet i rådata. For en eksempelkjøring var siste tidsstempel 2025-05-02 17:49:25+02:00, neste hele klokke-time satt til 2025-05-02 18:00:00+02:00 og prediksjonstidspunktet 2025-05-02 19:00:00+02:00. Tabellen viste da for eksempel at Florida Bybanestopp hadde 21 ledige sykler kl. 18:00 og en prediksjon på 18 sykler én time fram, mens Møllendalsplass lå på 4 nå og predikerte 7 én time senere.

- Forbedringer

Det er noen småting som jeg tror kunne vært forbedret i prosjektet, men som ikke har omfattende betydning for resultatet. For det første kunne jeg godt endret på variabelnavnet til `y_t_plus_1h`. Det er forklarende, men ganske langt, og kunne kanskje blitt endret til noe mer lesbart. For meg var det lettest å forholde meg til det samme gjennom hele prosjektet for å slippe misforståelser. For det andre har jeg trent «bare» 4 modeller. Jeg kunne godt ha trent flere, men jeg mener selv at modellutvalget mitt dekker de fleste mulighetene og gir et godt resultatet.

- Kildeliste

Tjenesten ChatGPT er brukt til å generere kode på følgende måte:

Generert oppsett for å finne riktig filbane i VS-code. Dette skal sørge for at koden kan kjøres på alle (windows) maskiner

Ledeteksten var "Jeg vil gjøre EDA i VS-code hvis det er mulig."

Tjenesten ChatGPT er brukt til å generere kode på følgende måte:

Gitt oppsett til en helsesjekk og laget en ferdig sjekk for stasjonsoversikt.

Ledeteksten var "Hva bør man inkludere i en helsesjekk for et slikt program?"

Tjenesten ChatGPT er brukt til å generere kode på følgende måte:

Generert kode for en graf som bruker IQR/2 som feilstopper.

Ledeteksten var "Hvordan kan jeg sammenligne avvik fra typisk time mot nedbør med IQR/2 som feilstopper?"

Tjenesten ChatGPT er brukt til å generere kode på følgende måte:

Lagre figurer som PNG-bilder.

Ledeteksten var "Hvordan kan man lagre figurer som PNG-filer?"

Tjenesten ChatGPT er brukt til å generere kode på følgende måte:

Generert kode for lagring av forhåndsversjon og generert en «base» for pipeline.py.

Ledeteksten var "Hvordan lager man en «pipeline» for å lagre en ferdig model.csv?"

Tjenesten ChatGPT er brukt til å generere kode på følgende måte:

Generert kode for å lagre den beste modellen med pickle på riktig måte.

Ledeteksten var "Hvordan lagrer man en modell ved å bruke pickle?"